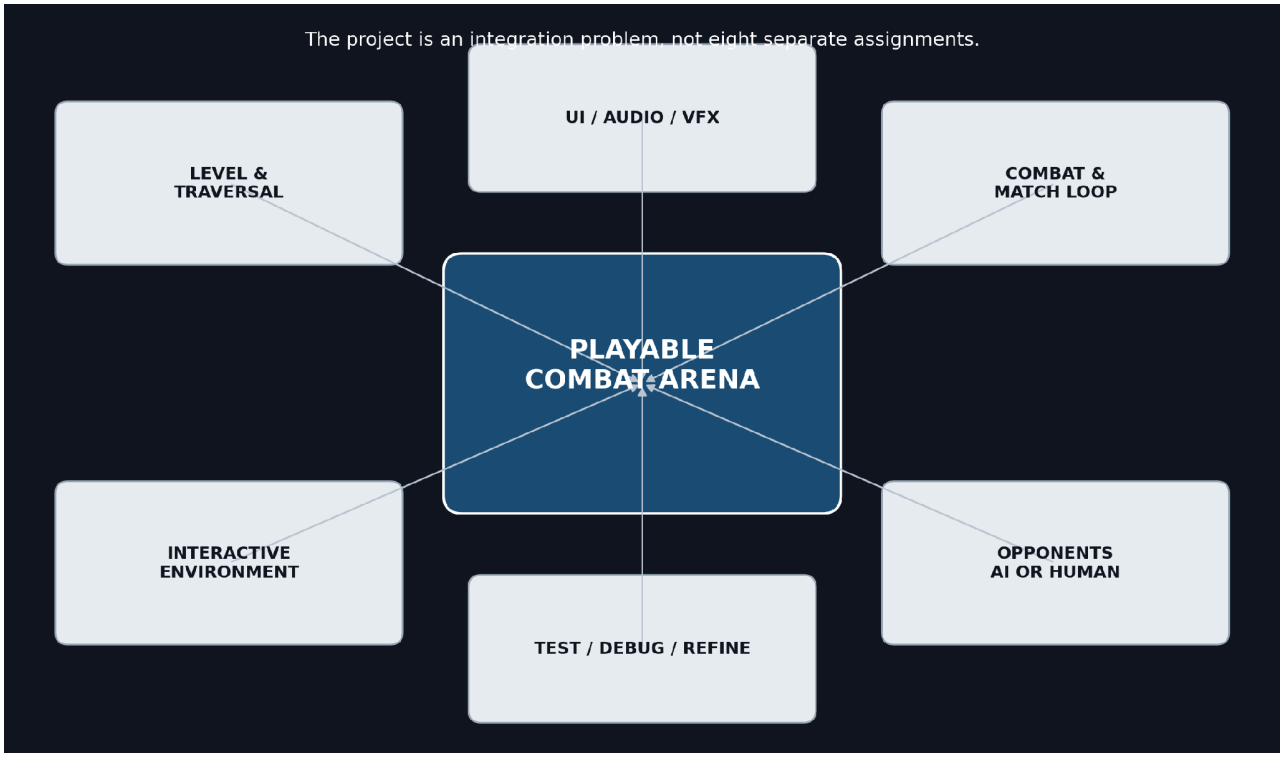
UNREAL ENGINE 5.8

GAME3023 — Unreal Engine: Development

TERM PROJECT BRIEF

Unreal Combat Arena

A semester-long, self-directed game development project



FORMAT	ENGINE	CLASS TIME	OUT-OF-CLASS
Individual or teams of up to 3	Unreal Engine 5.8	3 hours per week	Approx. 3–6 hours/week

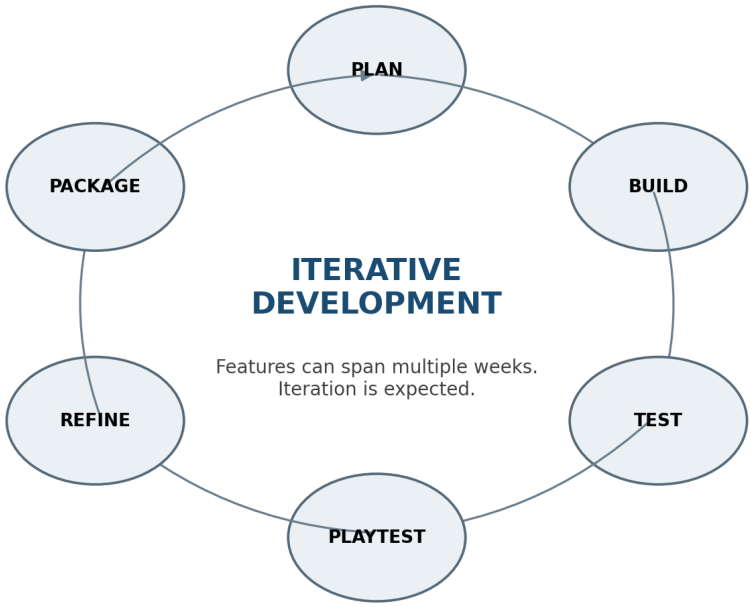
Draft project brief for instructor review and editing.

Project at a Glance

The central idea

Build one original Unreal combat arena over the entire semester. The same project grows from concept to blockout, first playable, Alpha, Beta, and final Release. Weekly topics are development focuses—not one-week deadlines for entire systems.

This course is designed as a transition from guided Unreal instruction to independent development. Students have already completed two Unreal Engine courses. GAME3023 asks them to use that prior knowledge, research unfamiliar details, select appropriate Unreal systems, integrate those systems, and solve the technical problems that arise in a larger project.



Project outcome

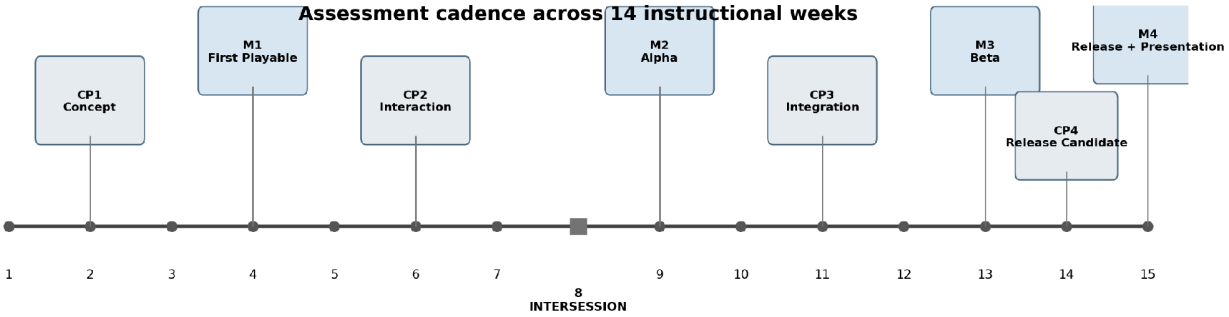
By the end of the term, each student or team will deliver a complete, packaged combat-arena experience that includes a navigable arena, combat, damage/death/respawn, a match loop, interactive environmental gameplay, hazards, player feedback, and functioning opponents. Opponents may be either AI-controlled or human players through simple Unreal networking.

What makes this a development course

- **Students are expected to research.** The instructor will introduce weekly goals and key Unreal systems, but will not provide a step-by-step solution for every feature.
- **Students are expected to integrate.** A feature is not complete because it works in isolation; it must work reliably inside the project.
- **Students are expected to iterate.** Arena layout, mechanics, balance, networking/AI, UI, and effects may all change after testing.
- **Students are expected to explain their work.** Documentation, tutorials, sample code, and AI assistance are permitted, but students must understand and defend what they submit.

Assessment pattern

There are only two assessment types throughout the project: smaller Checkpoints that confirm appropriate progress, and larger Milestones that establish a required state of playability or completion. This structure is intended to discourage last-minute development while still giving teams flexibility to let difficult features span several weeks.



Checkpoints verify progress. Milestones verify a defined state of playability/completion.

1. Project Overview

The major project for GAME3023 — Unreal Engine: Development is a semester-long Unreal Engine 5.8 project in which students design, develop, test, and refine an original combat arena experience. The project is intentionally constrained around an arena-combat structure so that students work toward comparable technical goals while still having substantial freedom in theme, layout, mechanics, hazards, visual language, and gameplay style.

The project should capture the spirit of classic arena games: a contained but interesting environment built around movement, combat, routes, verticality, environmental danger, pickups or resources, repeated play, and a complete match loop. The final product does not need to imitate a specific commercial game, and it does not need commercial-scale content.

> **Important scope principle**
>
> A smaller, stable, well-tested arena with coherent mechanics is stronger than a large project filled with unfinished systems. Scope may be reduced at any point if testing shows that the team cannot complete the planned feature set to an acceptable level.

1.1 Individual or team project

The project may be completed individually or in a team of two or three students. Students should select a scope appropriate to the size and experience of the development team.

- **Individual project:** Meets the required project baseline with a focused scope and demonstrates complete ownership of the implementation.
- **Two-person team:** May divide major areas of responsibility while both members remain familiar with the complete project.
- **Three-person team:** May pursue greater breadth, complexity, or polish, but should not add features simply to inflate scope.

Team members may specialize—for example, one student may focus more heavily on level gameplay, another on combat/match systems, and another on opponents/UI/technical polish—but every member is expected to understand the project as a whole. Individual contribution may be considered when determining grades for a shared project.

1.2 Creative direction

Students are encouraged to give the arena a strong identity. Suitable themes might include an industrial complex, orbital station, fantasy arena, experimental laboratory, underground facility, abstract simulation, abandoned city structure, training environment, or another concept approved by the instructor. Theme should support gameplay rather than create an excessive art-production burden.

1.3 Level construction philosophy

The arena should be constructed primarily inside Unreal Engine using native level-development tools. Students may use Geometry Brushes (the modern continuation of the BSP-style workflow familiar from older Unreal tools), Modeling Mode, Static Mesh primitives created in-engine, volumes, lights, and simple materials. External assets may supplement the project if permitted by the instructor and their licences, but imported environment art should not replace the level-development work being assessed.

Epic currently positions Geometry Brushes as useful for rapid level blockout and Modeling Mode as a more capable in-editor geometry workflow. For this course, students may retain simple brush/modeling geometry in the final project when it supports the learning goals; they are not being assessed as environment artists.

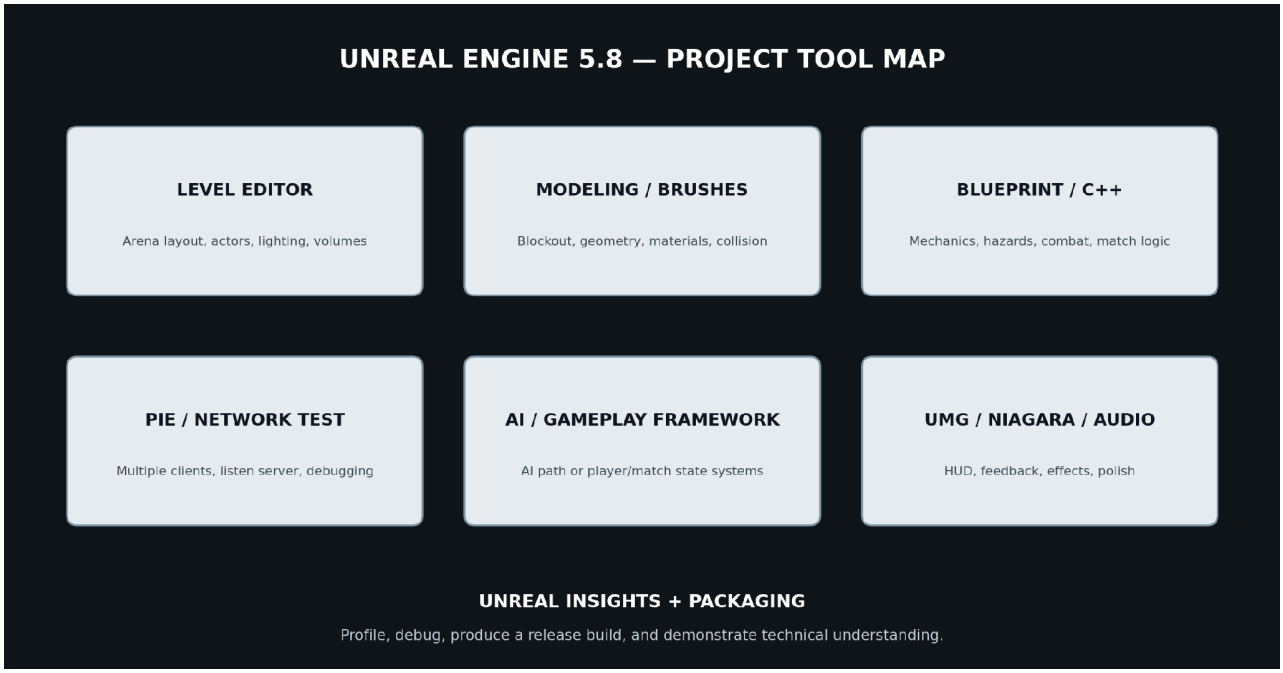
1.4 Baseline feature set

Requirement	Expected evidence / standard
Arena	One complete combat arena with intentional scale, traversal, routes, verticality and/or meaningful spatial variation.
Combat	At least one reliable combat method or weapon. Additional combat options are encouraged only if the core system is stable.
Health / damage	A working damage model with clear feedback.
Death / respawn	A complete return-to-play loop after death.
Opponents	Either functioning AI-controlled opponents or functioning networked human opponents.
Match loop	Scoring, kill tracking, timer, score limit, round state, or another clearly defined way to structure and conclude play.
Interactive environment	Multiple meaningful gameplay interactions such as doors, platforms, lifts, jump pads, switches, teleporters, moving obstacles, or similar mechanics.
Hazards	Environmental danger that affects play and is readable to players.
Pickups / resources	At least one meaningful pickup/resource category where appropriate (health, ammo, weapon, buff, energy, etc.).
UI / feedback	HUD or equivalent player feedback plus appropriate audio/visual responses for important events.
Release build	A stable packaged build that can be demonstrated without relying on the Unreal Editor.

1.5 What is not required

- Original professional-quality 2D or 3D art.
- A large open world or multiple finished maps.

- Commercial matchmaking, accounts, backend services, anti-cheat, or dedicated-server infrastructure.
- Both AI and human multiplayer opponents. One opponent path is sufficient.
- A large arsenal or complicated inventory system unless it directly supports the project concept.
- Advanced animation, cinematic sequences, or narrative content unrelated to the combat-arena goals.
- A completely original implementation of systems Unreal already provides. Appropriate use of engine frameworks is expected.



Official Unreal Engine 5.8 documentation

- UE 5.8 Documentation
- Level Editor
- Geometry Brush Actors (BSP)
- Modeling Mode
- Modeling Tools
- Materials
- Gameplay Framework

2. AI-Assisted Development and External Resources

AI usage is allowed throughout the term project

AI-assisted development is permitted for all Checkpoints and Milestones in this project, subject to current college requirements for disclosure, attribution, and academic integrity.

Students are encouraged to responsibly use AI-assisted development tools as part of their workflow for research, debugging, prototyping, documentation, and code or Blueprint assistance. AI-generated solutions must be understood, tested, integrated, and appropriately modified by the student. Students remain responsible for the functionality, quality, and technical understanding of all submitted work.

This allowance is intentionally broad. Students may also use official documentation, API references, tutorials, examples, sample projects, developer forums, technical articles, videos, and other legitimate development resources. The purpose of the course is not to test whether a student can work without

references; it is to test whether the student can use available resources to successfully develop, integrate, troubleshoot, and explain a working Unreal project.

2.1 What students remain responsible for

- Understanding the major systems they submit.
- Testing generated or copied solutions rather than assuming they are correct.
- Adapting solutions to the architecture and requirements of their own project.
- Debugging failures and integration problems.
- Complying with licences and attribution requirements for third-party assets or code.
- Disclosing AI use when required by current course/college policy.
- Answering technical questions during Checkpoints, Milestones, and the final presentation.

Technical defence rule

If a significant feature works but the student cannot explain how it works, why it was implemented that way, or how it was tested, the instructor may reduce the technical assessment for that feature.

3. Weekly Class Notes and Instructor Support

Short Class Notes will be generated for the major development phases of the project. These notes are intended to support independent development rather than replace it with full tutorials. A typical class will include a brief instructor introduction, demonstration, review, or troubleshooting discussion followed by approximately 2.5 hours of supervised project development.

Class Notes may include:

- The development goal for the current phase.
- Relevant Unreal Engine classes, tools, editor panels, or workflows to investigate.
- Recommended official documentation and examples.
- Common implementation mistakes or debugging approaches.
- Questions teams should be able to answer at the next Checkpoint or Milestone.
- Optional extension ideas for teams that are ahead of schedule.

A weekly topic does not imply that a complete feature must be finished during that week. Major systems may span multiple weeks, and earlier systems are expected to continue evolving throughout the project.

4. Semester Development Schedule

Week	Development focus	Expected activity / assessment
1	**Project kickoff**	Project brief, team formation, Unreal project setup, arena research, initial concept and scope.
2	**Concept & planning**	Arena plan, routes, spaces, technical feature ideas. Checkpoint 1.
3	**Blockout I**	Build major arena spaces; scale, traversal, collision, primary routes.
4	**Blockout II / first playable**	Refine layout, basic visual language, initial interaction. Milestone 1.
5	**Interactive environment I**	Reusable mechanics such as doors, platforms, lifts, switches, jump pads, teleporters.

Week	Development focus	Expected activity / assessment
6	**Interactive environment II**	Hazards, damage foundations, combat progress. Checkpoint 2.
7	**Combat & match development**	Weapons/attacks, health, death, respawn, pickups, scoring/match-state progress.
8	**INTERSESSION**	No scheduled instructional class.
9	**Opponent integration / Alpha**	AI opponents OR networked human opponents; complete core game loop. Milestone 2.
10	**Feature development**	Match systems, pickups, opponent refinement, environmental gameplay, integration.
11	**Player feedback & integration**	HUD, audio, VFX, readable game state. Checkpoint 3.
12	**Playtesting & arena iteration**	Structured playtesting, balance, spawn/route/pickup iteration, bug tracking.
13	**Beta**	Feature complete; focus shifts to bugs, balance, stability and packaging. Milestone 3.
14	**Release candidate**	Debugging, profiling, packaging, technical cleanup. Checkpoint 4.
15	**Release & presentation**	Final packaged build, live demonstration, technical discussion and reflection. Milestone 4.

5. Assessment Summary

Checkpoints are intentionally smaller than Milestones. They assess whether the project is moving in the right direction, whether the team can demonstrate meaningful progress, and whether the remaining plan is realistic. Milestones assess a defined state of the game.

Assessment	Due	Weight	AI Usage
Checkpoint 1 — Concept & Development Plan	Week 2	0%	Allowed
Milestone 1 — First Playable	Week 4	20%	Allowed
Checkpoint 2 — Interactive Gameplay Progress	Week 6	0%	Allowed
Milestone 2 — Alpha	Week 9	30%	Allowed
Checkpoint 3 — Feature Integration	Week 11	0%	Allowed
Milestone 3 — Beta	Week 13	20%	Allowed
Checkpoint 4 — Release Candidate Review	Week 14	0%	Allowed
Milestone 4 — Release & Presentation	Week 15	30%	Allowed

Checkpoint vs. Milestone

Checkpoint = show credible progress, identify current problems, and demonstrate that the next phase is achievable.
Milestone = demonstrate the required functional state of the game by the due date.

6. Checkpoint 1 — Concept & Development Plan

DUE	WEIGHT	TYPE
Week 2	**0%**	**Checkpoint**

Establish a viable arena concept, development plan, and Unreal project before significant production begins.

Checkpoint 1 is intentionally lightweight. The goal is not a large formal Game Design Document. Students should demonstrate that they have chosen an achievable idea, thought about how the arena will play, created the Unreal project, and identified the first technical risks.

Requirement	Expected evidence / standard
Project concept	A concise explanation of the arena theme, intended combat experience, and the main idea that makes the arena interesting.
Arena plan	A readable paper sketch, digital sketch, diagram, top-down plan, or equivalent showing the intended major spaces and connections.
Routes and flow	Identification of primary routes, alternate routes, possible vertical spaces, encounter areas, and likely spawn regions.
Feature ideas	A preliminary list of environmental mechanics, hazards, pickups/resources, combat ideas, and match rules. These are proposals—not promises that every idea will survive development.
Opponent direction	A preliminary choice between AI-controlled opponents and networked human opponents. The choice may change later with instructor approval.
Unreal project setup	Project created successfully, required plugins enabled where appropriate, basic folder/project organization established.
Team organization	For teams, a preliminary division of responsibilities and a method for sharing or versioning the project.
Scope check	A clear explanation of what will be considered the minimum viable version of the project if optional features must be cut.

Evidence / submission

- Concept and arena plan.
- Brief feature list and opponent plan.
- Working Unreal project setup.
- Live or recorded walkthrough if requested by the instructor.
- For teams: names, responsibilities, and collaboration/version-control plan.

Not required yet

- A finished level blockout.
- Combat.
- Working opponents.
- Final materials, lighting, UI, audio, or effects.
- A locked feature list.

Official Unreal Engine 5.8 documentation

Use these resources to plan the project and choose an in-editor level construction workflow.

- Projects and Templates
- Level Editor
- Geometry Brush Actors (BSP)
- Modeling Mode
- UE 5.8 Documentation

7. Milestone 1 — First Playable

DUE	WEIGHT	TYPE
Week 4	**20%**	**Milestone**

Prove that the planned arena exists in-engine as a complete navigable blockout that can already be evaluated through play.

The First Playable is primarily a level-development milestone. The arena should exist from end to end so the team can run through it, judge scale, discover navigation problems, and begin testing whether the spatial design can support combat. It is expected that the layout will continue changing after this milestone.

Requirement	Expected evidence / standard
Complete blockout	All major intended gameplay spaces and routes exist in the level at rough/blockout quality.
Player scale	Rooms, hallways, platforms, ramps, stairs, doorways, jumps and traversal spaces are sensibly scaled to the chosen player character.
Traversal	The player can navigate the arena without major unintended blockers or dead ends.
Routes / spatial choices	The layout contains meaningful circulation rather than functioning as a single linear corridor.
Verticality / variation	Height variation, upper/lower routes, ramps, platforms, drops, overlooks, or another form of spatial variation is included where appropriate.
Spawn planning	Preliminary PlayerStart/opponent-spawn areas are placed or clearly identified.
Collision	Major level collision behaves correctly enough for reliable movement and testing.
Basic visual language	Simple materials, colours, lighting, signage, or other cues begin differentiating spaces and helping navigation.
Initial interaction	At least one interactive/environmental feature is functional or substantially under development.
Project organization	Level and project assets are organized clearly enough to support continued team development.

Evidence / submission

- Playable Unreal project showing the entire arena blockout.
- Brief guided walkthrough of major spaces, routes and planned gameplay zones.
- Demonstration of player scale/collision and at least one early interactive element.
- Short explanation of changes made since the original plan and why.

Not required yet

- Finished combat or weapon systems.
- Functional opponents.
- Final scoring/match logic.
- Polished lighting or art.
- Finished UI, audio or VFX.
- A layout that will never change again.

Official Unreal Engine 5.8 documentation

Epic's Geometry Brush page explicitly describes blockout → playtest → modify as an early level-design workflow. Modeling Mode provides a more capable in-editor alternative.

- Geometry Brush Actors (BSP)
- Modeling Mode
- Modeling Mode Quick Start
- Modeling Tools
- Materials
- Playing and Simulating

8. Checkpoint 2 — Interactive Gameplay Progress

DUE	WEIGHT	TYPE
Week 6	**0%**	**Checkpoint**

Show that the arena is becoming an interactive game space and that the technical foundations for combat and hazards are advancing.

This Checkpoint is about direction and momentum. Features introduced in Weeks 5–6 do not all need to be complete. Students should be able to show meaningful progress on reusable environment mechanics, hazards, damage, and the beginning of combat, while also continuing to revise the level.

Requirement	Expected evidence / standard
Interactive mechanics	Multiple environmental interactions are functional or in active development: doors, platforms, lifts, switches, jump pads, teleporters, timed objects, rotating/moving hazards, etc.
Reusable implementation	At least some interactions are being implemented as reusable Blueprint Actors, Components, Interfaces, or comparable structures instead of relying entirely on one-off Level Blueprint logic.
Hazards	Environmental danger is implemented or clearly functional in prototype form.

Damage foundation	Health/damage logic is functioning or substantially implemented and can be demonstrated.
Combat progress	The main attack/weapon path is under development and can be demonstrated in at least an early form.
Arena iteration	The team has changed the arena in response to traversal testing, technical constraints, or instructor feedback.
Technical risk review	The student/team can identify the largest unresolved technical problems before Alpha and has a plan for addressing them.

Evidence / submission

- Live/recorded demonstration of current interactive mechanics.
- Demonstration of damage/hazard progress and combat progress.
- Short explanation of reusable Blueprint/C++ structure where applicable.
- Updated feature plan identifying what is complete, in progress, at risk, or deferred.

Not required yet

- A polished combat loop.
- Working multiplayer/AI opponent implementation.
- Final match UI or full scoring.
- All planned environmental features.

Official Unreal Engine 5.8 documentation

These references support reusable interaction systems, trigger/overlap logic, collision, and communication between gameplay Actors.

- Blueprints Visual Scripting
- Actor Communication
- Blueprint Interfaces
- Collision Overview
- Gameplay Framework Quick Reference

9. Milestone 2 — Alpha

DUE	WEIGHT	TYPE
Week 9	**30%**	**Milestone**

Demonstrate that the project has become an actual combat game: the core loop works, opponents work, and the arena supports repeated play.

Alpha is the most important functional milestone of the first half of the course. The project does not need to be polished, but the essential game must exist. A person should be able to enter the arena, encounter an opponent, fight, take damage, die, return to play, and continue participating in a match-like loop.

Requirement	Expected evidence / standard
Playable arena	Arena is complete enough to support meaningful movement and combat even though layout refinement may continue.
Combat	The primary combat method works reliably enough to fight an opponent.
Health and damage	Damage can be applied consistently and health/state changes are understandable.
Death	Player/opponent reaches a clear death or defeated state.
Respawn / return to play	The player can return to the game without restarting the entire application.
Opponent requirement	One opponent path is functional: AI-controlled opponents OR networked human opponents.
AI path	If AI is chosen, the opponent can navigate and meaningfully participate in combat rather than merely move through the arena.
Human-network path	If human multiplayer is chosen, multiple clients can join a simple session/test, move, fight, take damage, die/respawn, and observe the key replicated game state needed for the match.
Match state	Initial scoring, kill tracking, timer, score limit, round state, or comparable match logic is functional.
Environmental gameplay	Major interactive environment systems and hazards are present and affect play.
Pickups/resources	Initial pickup/resource functionality is included where appropriate.
Stability	The core loop can be demonstrated repeatedly without constant critical failures.

Opponent choice and online students

Students are not required to implement both AI and networking. For the human-network path, multi-client Play In Editor testing is acceptable for technical verification. Human playtesting is strongly encouraged when practical, but a student's grade should not depend on another person being available at the exact assessment time. For the AI path, students may build on their prior Unreal AI work and research appropriate Behavior Tree, Blackboard, navigation, perception, or related systems.

Evidence / submission

- A complete Alpha build/project state.
- Demonstration of the entire combat loop from spawn to combat to death/respawn.
- Demonstration of AI or networked opponent functionality.
- Demonstration of match-state/scoring progress.
- Brief technical explanation of the opponent architecture and one major combat or match system.

Not required yet

- Final UI or visual polish.
- Final balance.
- All optional features.
- Release-quality audio/VFX.

- Final optimization or packaging quality.

Official Unreal Engine 5.8 documentation

Use the networking references for human opponents, or the Behavior Tree references for AI opponents. GameMode/GameState/PlayerState are especially relevant to match and player-state design.

- Gameplay Framework
- Game Mode and Game State
- Multiplayer Programming Quick Start
- Play In Editor Multiplayer Options
- Testing Multiplayer
- Behavior Trees
- Behavior Tree User Guide

10. Checkpoint 3 — Feature Integration

DUE	WEIGHT	TYPE
Week 11	**0%**	**Checkpoint**

Show that the systems added after Alpha are becoming a cohesive player experience rather than remaining isolated technical features.

Checkpoint 3 occurs while the project is moving toward feature completion. The emphasis is integration: opponent systems, match state, pickups/resources, HUD, feedback, environment mechanics, audio, and VFX should increasingly work together. This is also a good point to cut features that are no longer realistic for Beta.

Requirement	Expected evidence / standard
Opponent refinement	AI/network behavior is more reliable and better integrated with combat, respawning and match systems.
Match systems	Scoring, match state, player state, timers/limits, and other core rules are approaching completion.
Pickups/resources	Gameplay resources are functional and their placement begins supporting level design and risk/reward.
HUD / UI	The player can read the game state that matters: health, score, ammo/resource, timer, respawn status, or equivalent.
Gameplay feedback	Important actions produce appropriate feedback using sound, VFX, animation, camera response, materials, UI, or another suitable method.
Environmental integration	Interactive level mechanics and hazards meaningfully interact with the combat experience rather than existing only as demonstrations.
Technical cleanup	The team is addressing integration bugs, poor dependencies, duplicated logic, fragile Blueprints, replication issues, or other technical debt.
Beta plan	A realistic list identifies what must be completed, cut, fixed, or tested before Week 13.

Evidence / submission

- Current playable build/project.
- Demonstration of HUD and player feedback.
- Demonstration of match-state and pickup/resource progress.
- Updated Beta task list with required work prioritized over optional additions.

Not required yet

- Final visual polish.
- Perfect balance.
- All bugs resolved.
- Final packaged release.

Official Unreal Engine 5.8 documentation

These resources support player-facing feedback and the integration of UI, effects, audio, and game-state data.

- UMG UI Designer Quick Start
- Niagara Quick Start
- Audio Engine Overview
- Game Mode and Game State
- Actor Communication

11. Milestone 3 — Beta

DUE	WEIGHT	TYPE
Week 13	**20%**	**Milestone**

Reach feature completion. After Beta, the team should primarily fix, balance, test, optimize and package—not invent major new systems.

Beta is a feature-complete build. All major required systems should be present and integrated. The project may still contain bugs, rough presentation, balance problems, or performance issues, but there should be no major missing system standing between the Beta and a credible final Release.

Requirement	Expected evidence / standard
Complete arena	The intended arena is fully playable and has undergone meaningful layout iteration.
Combat	Required combat systems are implemented and usable through repeated play.
Opponents	Chosen AI or human-network opponent solution is complete enough for sustained playtesting.
Complete game loop	Spawn/start → movement/combat → damage/death → respawn → scoring/match progression → match conclusion works.

Requirement	Expected evidence / standard
Interactive environment	Required interactive systems are implemented and meaningfully placed.
Hazards	Hazards are functional, readable and reasonably balanced.
Pickups/resources	Required resources are implemented and placed intentionally.
UI / information	Required gameplay information is visible and updates correctly.
Audio / VFX / feedback	Major game events communicate successfully to the player.
Playtesting	The project has been tested by people beyond the primary developer(s) where practical, or through structured repeat testing if peer access is limited.
Balance / iteration	The team can identify specific changes made because of testing.
Stability	No known critical blocker prevents normal play through the core loop.
Packaging test	The team has produced or is capable of producing a testable packaged build.

Feature lock

After Beta

New major features should normally stop. Teams should focus on bugs, usability, balance, technical cleanup, packaging, performance, visual/audio polish, and presentation readiness.

Evidence / submission

- Feature-complete Beta build/project.
- A current known-issues list.
- Evidence of structured playtesting and at least several specific changes made from results.
- Successful packaging test or evidence that packaging issues are actively being resolved.

Official Unreal Engine 5.8 documentation

Beta is where repeat testing, profiling and packaging become part of normal development rather than end-of-term emergencies.

- Playing and Simulating
- Testing Multiplayer
- Unreal Insights
- Packaging Your Project

12. Checkpoint 4 — Release Candidate Review

DUE	WEIGHT	TYPE
Week 14	**0%**	**Checkpoint**

Confirm that the project is truly close to release and that the remaining work is limited, prioritized, and achievable.

The Release Candidate should be near-final. Students should arrive with a build that could plausibly become the submitted Release after one more focused development cycle. The goal is to expose late technical risks—especially packaging, networking/AI reliability, collision, critical bugs, and performance—while there is still time to fix them.

Requirement	Expected evidence / standard
Known issues	Major remaining bugs/issues are identified and prioritized rather than discovered accidentally during the final presentation.
Critical blockers	Gameplay-breaking issues are resolved or have a credible immediate resolution plan.
Opponent reliability	AI/networking is stable enough for the final demonstration.
Level health	Collision, spawn, navigation, hazard, route, and major environment issues are resolved.
Performance / profiling	Obvious performance problems have been investigated and unnecessary expensive behavior has been reduced where appropriate.
Technical cleanup	Broken/unused systems, debugging leftovers, duplicate logic, naming/organization problems, or other obvious project-health issues are being addressed.
Packaging	A packaged build has been produced successfully or the remaining packaging problem is isolated and understood.
Presentation readiness	The student/team knows what will be demonstrated and which technical feature(s) will be explained.
Final task list	Remaining tasks fit realistically within the time before the final Release.

Evidence / submission

- Near-final project/build.
- Known-issues and priority list.
- Packaging test result.
- Brief performance/debugging evidence where appropriate.
- Final presentation plan and individual speaking responsibilities for teams.

Official Unreal Engine 5.8 documentation

Use these references to diagnose performance, verify networked behavior, and produce a reliable standalone build.

- Unreal Insights
- Packaging Your Project
- Build Operations
- Testing Multiplayer

13. Milestone 4 — Release & Presentation

DUE	WEIGHT	TYPE
Week 15	**30%**	**Milestone**

Deliver and defend the completed Unreal Combat Arena as a stable, playable, technically understood project.

The final Release represents the completed course project. It should feel like a finished student game rather than a collection of Unreal experiments. The instructor is evaluating not only whether features exist, but whether the arena, combat, opponent solution, match rules, interaction, feedback and technical implementation work together as one coherent experience.

Requirement	Expected evidence / standard
Arena design	Complete, readable, navigable, intentionally structured, and appropriately revised after playtesting.
Combat experience	Functional, understandable, stable, and appropriate to the arena.
Opponents	Chosen AI or networked-human opponent solution works reliably enough for a live demonstration.
Match loop	Rules, scoring/state, death/respawn and match progression form a complete playable loop.
Interactive environment	Mechanics contribute to movement, combat, risk/reward, timing, routes, or other meaningful gameplay.
Hazards	Functional, readable, and integrated into arena design.
Pickups/resources	Function correctly and are placed intentionally where used.
UI / player information	Necessary game state is clearly communicated.
Audio / VFX / feedback	Important game actions and states have appropriate feedback.
Technical quality	Project is organized, stable, and implemented with reasonable Unreal practices for the project scope.
Testing / iteration	Final product reflects evidence of playtesting, bug fixing, balancing and refinement.
Packaging	Standalone release build launches and functions correctly.
Creativity	The project demonstrates original design/implementation decisions rather than simply reproducing a single tutorial or sample.
Technical understanding	Student(s) can explain the systems, tools, sources, AI assistance and decisions behind the work.

Final submission package

1. **A packaged playable build** for the platform specified by the instructor (normally Windows for this project).

2. A short README containing project title, team members, controls, how to launch/play, opponent mode (AI or networked), and any known limitations.
3. A brief contribution summary for team projects identifying each member's primary technical work.
4. Any AI-use or external-resource disclosure required by current college/course policy.
5. Presentation support material if used (slides are optional unless required by the instructor).

Final presentation requirements

Recommended presentation length

Approximately 5 minutes per project in video form displayed in-class, followed by instructor questions. Students must be present to answer questions. The instructor may adjust the time based on section size.

Presentation component	Requirement
Project introduction	State the arena concept, theme, team members, chosen opponent path, and intended gameplay experience.
Gameplay	Demonstrate the actual packaged build where practical, not only editor footage or slides.
Arena tour	Show major areas, routes, verticality/spatial choices, and explain at least one level-design decision.
Combat demonstration	Demonstrate combat, damage, death/respawn, pickups/resources, and match progression.
Opponent demonstration	Show functioning AI combat or functioning human-networked play. For networking, demonstrate at least two connected player instances where practical.
Interactive environment	Demonstrate important doors/platforms/lifts/jump pads/teleporters/hazards/other mechanics and explain their gameplay purpose.
Technical deep dive	Explain at least one significant Blueprint/C++/Unreal system in enough detail to demonstrate understanding of architecture and data flow.
Development challenge	Describe a meaningful technical/design problem and how it was researched, debugged and resolved.
Iteration evidence	Show or explain at least one meaningful change made because of playtesting or feedback.
AI and sources	Briefly describe how AI-assisted tools, documentation, tutorials, sample code, or other resources contributed to development.
Team contribution	Each team member identifies major areas of contribution. All members participate in the presentation.
Reflection	Explain what would be improved, extended, or redesigned with additional development time.
Questions	Be prepared to answer technical questions about any major project system, not only the feature personally demonstrated.

Official Unreal Engine 5.8 documentation

The final demonstration should use a standalone packaged build whenever practical. Epic's packaging guide includes a Windows packaging walkthrough.

- Packaging Your Project
- Sharing and Releasing Projects
- Testing Multiplayer
- UE 5.8 Documentation

14. Teamwork and Individual Accountability

Team projects should be collaborative, but collaboration does not require that every student write every Blueprint or touch every level object. Teams may divide responsibilities according to strengths and project needs. What matters is that work is meaningfully distributed, integrated, and understood.

- **Reasonable specialization:** level blockout and environment gameplay; combat/character systems; networking or AI; UI/feedback; technical integration; testing/packaging. Roles may change over the term.
- **Shared responsibility:** all team members should understand the core game loop, be able to open and navigate the project, and understand how major systems communicate.
- **Evidence of contribution:** source control history, task boards, change logs, working notes, Blueprint ownership, code commits, build responsibility, or other reasonable evidence may be used if contribution is unclear.
- **Individual adjustment:** the instructor may adjust an individual grade where participation, contribution, or technical understanding differs substantially within a team.

Recommended team workflow

1. Agree on the project baseline before adding optional features.
2. Keep a shared task list that clearly distinguishes required work from optional polish.
3. Assign an owner to each major feature, but require another member to test it.
4. Integrate frequently rather than allowing each member to develop isolated branches until the end.
5. Produce regular playable builds, especially before Milestones.
6. Use Checkpoints to revise scope honestly when a planned feature is consuming too much time.

15. Arena Design and Gameplay Guidance

The arena does not need to demonstrate professional competitive level design, but it should show deliberate thought. Students should expect the layout to change after actual combat begins because a space that feels good to walk through can behave very differently when players are shooting, chasing, retreating, using hazards, or controlling valuable areas.

Design question	What to consider
Scale	Can players read and navigate spaces comfortably? Are corridors, platforms and vertical distances appropriate to movement speed and combat range?
Flow	Can players move through multiple routes? Are there natural loops rather than repeated dead ends?
Sightlines	Can one location see too much of the map? Are long and short combat spaces intentionally distributed?

Design question	What to consider
Verticality	Do upper/lower routes or elevation changes create meaningful movement choices rather than only decoration?
Spawn safety	Can players re-enter the match without being immediately trapped or killed repeatedly?
Pickup placement	Do resources encourage movement and risk/reward rather than simply appearing beside spawn points?
Interactive mechanics	Do moving platforms, doors, teleporters, lifts or hazards change decision-making?
Readability	Can players identify hazards, exits, pickups, important landmarks and interactive objects?
Balance	Are there dominant positions, weapons, routes or hazards that make other choices irrelevant?
Replayability	Does repeated play continue producing interesting movement and combat situations?

16. Playtesting, Debugging and Iteration

Playtesting is part of development, not something reserved for the final two weeks. Students should repeatedly test the arena from the beginning of blockout. Testing should become more structured as the project approaches Alpha and Beta.

- Run the map from multiple spawn locations.
- Test routes in both directions and under pressure.
- Test hazards with intentional and accidental entry.
- For networking, test as listen server and client and watch for state that appears correct on only one machine/client.
- For AI, test navigation, target acquisition, path failures, repeated deaths/respawns, and unusual player locations.
- Record bugs rather than relying on memory.
- After each playtest, choose a small number of high-value changes and retest them.
- Keep optional features below stability and core-loop fixes in priority.

Playtesting evidence for Beta

Teams should be able to name specific problems discovered during playtesting and specific changes made as a result. “We tested it and it seemed fine” is not sufficient evidence of iteration.

17. Official Unreal Engine 5.8 Documentation Reference

The following links are included throughout this brief and are collected here for convenience. Epic may update page content or redirect documentation as Unreal Engine versions change. Students should confirm that the page header corresponds to the version used in class when version-specific behavior matters.

Level building and environment

- Level Editor

- Geometry Brush Actors (BSP)
- Modeling Mode
- Modeling Mode Quick Start
- Modeling Tools
- Materials
- Material Editor Guide
- Playing and Simulating

Gameplay scripting and architecture

- Blueprints Visual Scripting
- Actor Communication
- Blueprint Interfaces
- Collision Overview
- Gameplay Framework
- Gameplay Framework Quick Reference
- Game Mode and Game State
- C++ API Reference

Opponents and multiplayer

- Multiplayer Programming Quick Start
- Play In Editor Multiplayer Options
- Testing Multiplayer
- Behavior Trees
- Behavior Tree User Guide

UI, feedback and polish

- UMG UI Designer Quick Start
- Niagara Quick Start
- Audio Engine Overview

Testing, profiling and release

- Unreal Insights
- Packaging Your Project
- Build Operations
- Sharing and Releasing Projects